Ticketer

Une plateforme moderne de billetterie et de gestion de concerts.



Démo Live : https://ticketer-ife6.onrender.com

À propos du projet

Ticketer a été conçu dans le cadre d'un projet de formation complet. Il répond à un besoin concret : centraliser et simplifier la réservation de billets de concerts pour les fans, tout en offrant aux organisateurs un panneau d'administration intuitif pour gérer leurs événements.

L'application permet d'explorer un catalogue d'artistes dynamique, de réserver ses places en quelques clics, et de retrouver ses billets instantanément.

Fonctionnalités clés

Pour les utilisateurs :

- **Interface Responsive :** Navigation fluide entre Desktop et Mobile (carrousels interactifs).
- **Génération de Billets :** Création automatique d'un QR code unique pour chaque réservation.
- **Notifications :** Soyez alerté instantanément lorsqu'un de vos artistes favoris ajoute une nouvelle date de concert.

Pour les administrateurs :

- **Gestion centralisée :** Création et mise à jour des groupes/artistes, des genres musicaux et des concerts.
- **Contrôle d'accès :** Gestion complète des utilisateurs enregistrés et de leurs réservations.
- **Stockage Cloud :** Upload et hébergement sécurisé des images d'artistes et de concerts via Cloudinary.

Stack Technique

- **Backend :** NestJS (Node.js)
- **Frontend :** Moteur de templates Handlebars (.hbs), Tailwind CSS
- **Base de données :** MongoDB avec Mongoose
- **Stockage de médias :** Cloudinary

Installation et Configuration

Prérequis

- Node.js (v18 ou supérieur recommandé)
- Une base de données MongoDB (locale ou Atlas)
- Un compte Cloudinary pour le stockage des images

Instructions

1. **Cloner le dépôt**

```
git clone https://github.com/votre-nom-utilisateur/ticketer.git
cd ticketer
```

2. **Installer les dépendances**

```
npm install
```

3. **Configurer les variables d'environnement** Créez un fichier `.env` à la racine du projet et ajoutez les variables suivantes :

```
# Base de données
MONGODB_URI=votre_url_mongodb_atlas_ou_local

# Configuration Cloudinary
CLOUDINARY_CLOUD_NAME=votre_cloud_name
CLOUDINARY_API_KEY=votre_api_key
CLOUDINARY_API_SECRET=votre_api_secret

# Authentification
SESSION_SECRET=votre_secret_de_session
```

4. **Lancer l'application (Mode Développement)**

```
npm run start:dev
```

   L'application sera accessible sur `http://localhost:3000`.

Mention

Ce projet a été réalisé dans un but éducatif et de formation. N'hésitez pas à l'explorer pour découvrir l'implémentation d'une architecture MVC avec NestJS.